

TECHNICAL LEARNING GUIDE

Model Context Protocol (MCP)

How it works, how to set it up on Windows, and how it fits into the two-agent loan prototype

Outcome: By the end, you will have a working MCP server, a visual Inspector test, a Python MCP client, and a clear integration path for agents, Redis, OKF, and PostgreSQL.

Prepared for the loan-agent proof of concept | Windows | Python 3.12

Quick reference

Component	Role
LLM / Agent	Reasons about an event and chooses an allowed tool.
MCP Host	Runs the agent and manages MCP client connections.
MCP Client	Discovers and calls one MCP server.
MCP Server	Exposes typed tools, resources, and prompts.
Redis Streams	Delivers workflow events to the correct agent.
Redis Iris	Stores or retrieves agent context and memory.
OKF wiki	Stores curated policies and domain knowledge.
PostgreSQL	Stores authoritative loan state and audit history.

1. What MCP is

MCP stands for Model Context Protocol. It is a standard interface through which an AI application can discover and call external capabilities. MCP does not provide the intelligence, database, event queue, or business policy. It defines how a host and server describe capabilities and exchange requests and results.

```
LLM / Agent = brain
MCP Server = controlled tool box
PostgreSQL = permanent business data
Redis = workflow events
OKF = curated business knowledge
```

Core rule: For the loan prototype, an agent should never connect directly to PostgreSQL. Use Agent -> MCP tool -> validated domain service -> PostgreSQL.

2. Core participants

MCP host

The host is the complete AI application. In this project, the Python worker that runs the Verification Agent or Decision Agent is the host. It sends prompts to the model, exposes tool schemas, manages MCP clients, applies approval rules, and returns tool results to the model.

MCP client

The MCP client lives inside the host. It opens one stateful connection to one MCP server, negotiates capabilities, lists available tools, calls tools, and receives structured results.

MCP server

The server is an ordinary program that exposes controlled capabilities. It does not need an LLM. Your Loan MCP server should contain deterministic validation and call a LoanService that owns database access.

3. Tools, resources, and prompts

Primitive	Meaning	Loan example
Tool	An executable, schema-defined operation.	get_loan, save_verification
Resource	Read-only data supplied as context.	loan schema or policy file
Prompt	Reusable instruction template.	verify-loan workflow prompt

For the first prototype, focus on tools. Add resources only after the tool flow works. Your OKF files can initially be loaded by the application rather than exposed as MCP resources.

4. What happens during a tool call

- › The host connects to the MCP server and sends initialize.
- › The server returns its supported capabilities.
- › The client requests tools/list.
- › The server returns tool names, descriptions, and JSON input schemas.
- › The host includes those tool definitions in the model request.
- › The model requests a tool such as get_loan.
- › The host validates approval requirements and sends tools/call.
- › The MCP server validates the input and invokes the domain service.
- › The server returns structured content to the host.
- › The host gives the result back to the model for the next step.

```
{
  "jsonrpc": "2.0",
  "id": 10,
  "method": "tools/call",
  "params": {
    "name": "get_loan",
    "arguments": {"loan_id": "loan-101"}
  }
}
```

5. Transport choice

Transport	Use it when	Recommendation
STDIO	The host starts a local MCP subprocess.	Fastest isolated experiment.
Streamable HTTP	Multiple agents connect to a shared server.	Use for this loan prototype.

STDIO warning: Do not print logs to stdout in an STDIO server because stdout carries JSON-RPC messages. Send logs to stderr or a file. Normal logging is safe with Streamable HTTP.

6. Windows setup

6.1 Install uv and Python

```
winget install --id=astral-sh.uv -e

# Close and reopen PowerShell
uv --version
uv python install 3.12
uv python list
```

If winget is unavailable, use the official uv installer:

```
powershell -ExecutionPolicy Bypass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

6.2 Create the MCP practice project

```
mkdir loan-mcp-practice
cd loan-mcp-practice
uv init
uv venv --python 3.12
uv add "mcp[cli]>=1.27,<2"
```

Version note: At the time this guide was prepared, MCP Python SDK v1 was stable and v2 was prerelease. Pinning below v2 avoids unexpected breaking changes during the short prototype.

7. Build the first Loan MCP server

Create `loan_mcp_server.py`. This first version uses in-memory data so that MCP can be tested before PostgreSQL is introduced.

```
from typing import Literal

from mcp.server.fastmcp import FastMCP

mcp = FastMCP(
    "Loan MCP Server",
    instructions="Controlled access to loan applications.",
    json_response=True,
    host="127.0.0.1",
    port=8001,
)

loans = {
    "loan-101": {
        "id": "loan-101",
        "monthly_income": 60000,
        "credit_score": 760,
        "requested_amount": 200000,
        "documents_complete": True,
        "status": "SUBMITTED",
        "version": 1,
    }
}

ALLOWED_TRANSITIONS = {
    "SUBMITTED": {"VERIFYING"},
    "VERIFYING": {"VERIFIED", "DOCUMENTS_REQUIRED"},
    "DOCUMENTS_REQUIRED": {"VERIFYING"},
    "VERIFIED": {"ASSESSING"},
    "ASSESSING": {"RECOMMENDED"},
    "RECOMMENDED": {"MANUAL_REVIEW", "REJECTED"},
    "MANUAL_REVIEW": {"APPROVED", "REJECTED"},
    "APPROVED": {"SANCTIONED"},
    "REJECTED": set(),
    "SANCTIONED": set(),
}
```

```

@mcp.tool()
def get_loan(loan_id: str) -> dict:
    """Get one loan application by its unique ID."""
    loan = loans.get(loan_id)
    if loan is None:
        return {"success": False, "error": "LOAN_NOT_FOUND"}
    return {"success": True, "loan": loan}

@mcp.tool()
def calculate_eligibility(loan_id: str) -> dict:
    """Calculate a deterministic eligibility recommendation."""
    loan = loans.get(loan_id)
    if loan is None:
        return {"success": False, "error": "LOAN_NOT_FOUND"}

    if not loan["documents_complete"]:
        recommendation = "DOCUMENTS_REQUIRED"
        reason = "Required documents are incomplete."
    elif loan["credit_score"] < 550:
        recommendation = "REJECT"
        reason = "Credit score is below the demo minimum."
    elif (
        loan["credit_score"] >= 750
        and loan["requested_amount"]
        <= loan["monthly_income"] * 12 * 5
    ):
        recommendation = "APPROVE"
        reason = "Application satisfies the demo rules."
    else:
        recommendation = "MANUAL_REVIEW"
        reason = "An authorized person must review the loan."

    return {
        "success": True,
        "loan_id": loan_id,
        "recommendation": recommendation,
        "reason": reason,
    }

@mcp.tool()
def change_loan_status(
    loan_id: str,
    new_status: Literal[
        "VERIFYING", "VERIFIED", "DOCUMENTS_REQUIRED",
        "ASSESSING", "RECOMMENDED", "MANUAL_REVIEW",
        "APPROVED", "REJECTED", "SANCTIONED",
    ],
    expected_version: int,
    reason: str,
) -> dict:
    """Change a loan state after validating transition and version."""
    loan = loans.get(loan_id)

    if loan is None:
        return {"success": False, "error": "LOAN_NOT_FOUND"}
    if loan["version"] != expected_version:
        return {
            "success": False,
            "error": "VERSION_CONFLICT",
            "current_version": loan["version"],
        }

    current = loan["status"]
    allowed = ALLOWED_TRANSITIONS.get(current, set())
    if new_status not in allowed:
        return {
            "success": False,
            "error": "INVALID_STATE_TRANSITION",
            "current_status": current,
            "allowed_statuses": sorted(allowed),
        }

    loan["status"] = new_status
    loan["version"] += 1

```

```

    return {
        "success": True,
        "loan_id": loan_id,
        "previous_status": current,
        "current_status": new_status,
        "version": loan["version"],
        "reason": reason,
    }

if __name__ == "__main__":
    mcp.run(transport="streamable-http")

```

8. Run and inspect the server

8.1 Start the server

```
uv run loan_mcp_server.py
```

The endpoint is:

```
http://127.0.0.1:8001/mcp
```

This is a protocol endpoint, so it may not display a normal webpage when opened directly.

8.2 Install Node.js for MCP Inspector

```

winget install OpenJS.NodeJS.LTS

# Restart PowerShell
node --version
npm --version

```

8.3 Launch Inspector

```
npx -y @modelcontextprotocol/inspector
```

- > Open the local Inspector URL printed in PowerShell.
- > Choose Streamable HTTP as the transport.
- > Enter `http://127.0.0.1:8001/mcp`.
- > Connect and open the Tools tab.
- > Run `get_loan` with `loan_id` `loan-101`.
- > Run `calculate_eligibility` with the same loan ID.
- > Run `change_loan_status` with `new_status` `VERIFYING` and `expected_version` `1`.

```

{
  "loan_id": "loan-101",
  "new_status": "VERIFYING",
  "expected_version": 1,
  "reason": "Verification process started"
}

```

9. Call the server from Python

Create `test_client.py`:

```

import asyncio

from mcp import ClientSession
from mcp.client.streamable_http import streamablehttp_client

```

```

async def main() -> None:
    url = "http://127.0.0.1:8001/mcp"

    async with streamablehttp_client(url) as (
        read_stream,
        write_stream,
    ):
        async with ClientSession(
            read_stream,
            write_stream,
        ) as session:
            await session.initialize()

            tools = await session.list_tools()
            for tool in tools.tools:
                print(f"{tool.name}: {tool.description}")

            result = await session.call_tool(
                "get_loan",
                {"loan_id": "loan-101"},
            )
            print(result)

if __name__ == "__main__":
    asyncio.run(main())

uv run test_client.py

```

10. Connect an AI agent

Install the OpenAI Agents SDK:

```

uv add openai-agents
$env:OPENAI_API_KEY="your-api-key"

```

Secret handling: Never commit an API key to Git. Use a .env file excluded by .gitignore or a secrets manager.

Create `verification_agent.py`:

```

import asyncio

from agents import Agent, Runner
from agents.mcp import MCPServerStreamableHttp

async def main() -> None:
    async with MCPServerStreamableHttp(
        name="Loan MCP",
        params={
            "url": "http://127.0.0.1:8001/mcp",
            "timeout": 15,
        },
        cache_tools_list=True,
        max_retry_attempts=2,
    ) as loan_server:
        agent = Agent(
            name="Loan Verification Agent",
            instructions=(
                "Always call get_loan first. "
                "Never approve or sanction a loan. "
                "Only begin verification and report missing data."
            ),
            mcp_servers=[loan_server],
        )

        result = await Runner.run(
            agent,
            "Start verification for loan-101.",
        )
        print(result.final_output)

```

```
if __name__ == "__main__":
    asyncio.run(main())

uv run verification_agent.py
```

11. How MCP fits the loan architecture

```
EVENT PATH
FastAPI / PostgreSQL -> Redis Stream -> Agent

ACTION PATH
Agent -> MCP Client -> Loan MCP Server -> LoanService -> PostgreSQL

CONTEXT PATH
OKF files + Redis Iris Agent Memory -> Agent
```

Question	Component that answers it
What happened, and which agent should run?	Redis Streams
What action may the agent perform?	MCP tools
What is the current authoritative state?	PostgreSQL
What policies should the agent understand?	OKF wiki
What happened earlier in this workflow?	Redis Iris Agent Memory

Important: MCP does not replace Redis, and Redis does not replace MCP. Redis triggers work; MCP controls actions.

12. Replace in-memory data with PostgreSQL

Once every MCP tool works in Inspector, replace the loans dictionary with a LoanService. Keep the MCP layer thin.

```
class LoanService:
    def get_loan(self, loan_id: str) -> dict:
        ...

    def calculate_eligibility(self, loan_id: str) -> dict:
        ...

    def change_status(
        self,
        loan_id: str,
        new_status: str,
        expected_version: int,
        reason: str,
    ) -> dict:
        ...

@mcp.tool()
def get_loan(loan_id: str) -> dict:
    """Get an authorized loan application by ID."""
    return loan_service.get_loan(loan_id)

MCP tool = protocol interface
LoanService = business rules
```

SQLAlchemy = database access

13. Safe MCP tool design

Good loan tools

- `get_loan(loan_id)`
- `get_loan_history(loan_id)`
- `save_verification(loan_id, result, expected_version)`
- `calculate_eligibility(loan_id)`
- `save_recommendation(loan_id, recommendation, expected_version)`

Avoid broad or dangerous tools

- `execute_sql(sql)`
- `update_any_table(table, values)`
- `delete_loan(loan_id)`
- `sanction_loan_without_approval(loan_id)`

Every write tool should validate

- Authentication and the caller's role.
- The current loan state and requested next state.
- The expected record version to prevent lost updates.
- An idempotency key to prevent duplicate event processing.
- Input and output schemas.
- Human approval for approval or sanction actions.
- An audit record describing who changed what and why.

14. Windows troubleshooting

Problem	Check or fix
uv is not recognized	Restart PowerShell, then run <code>Get-Command uv</code> .
Activation is blocked	Use <code>uv run</code> , or set execution policy for the current process only.
npm is not recognized	Install Node.js LTS and restart PowerShell.
Port 8001 is busy	Run <code>Get-NetTCPConnection -LocalPort 8001</code> or change the MCP port.
Inspector cannot connect	Verify server, transport, URL, port, and Windows Firewall.
STDIO gives JSON errors	Remove stdout print calls; send logs to stderr.
Tool is missing	Reconnect Inspector and run <code>tools/list</code> after restarting the server.

15. Recommended learning sequence

- › Understand host, client, server, tool, resource, and prompt.
- › Run the in-memory MCP server.
- › Test every tool through MCP Inspector.
- › Call one tool from the Python MCP client.
- › Connect only the Verification Agent.
- › Replace the dictionary with PostgreSQL and LoanService.
- › Connect the worker to Redis Streams.
- › Add the Decision Agent.
- › Add Redis Iris memory and OKF policy loading.
- › Add approval, authorization, audit, retries, and idempotency.

Best development rule: Do not combine all technologies on the first attempt. Make one MCP tool work in Inspector, then add one layer at a time.

Official references

- MCP architecture: <https://modelcontextprotocol.io/docs/learn/architecture>
- Build an MCP server: <https://modelcontextprotocol.io/docs/develop/build-server>
- MCP server concepts: <https://modelcontextprotocol.io/docs/learn/server-concepts>
- MCP Inspector: <https://modelcontextprotocol.io/docs/tools/inspector>
- Python SDK: <https://github.com/modelcontextprotocol/python-sdk>
- OpenAI Agents SDK MCP integration: <https://openai.github.io/openai-agents-python/mcp/>
- uv Windows installation: <https://docs.astral.sh/uv/getting-started/installation/>